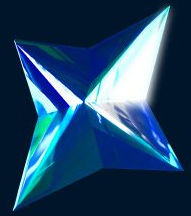
BRO Academy



Templating: twig data

Lesson 8

Menti

Lesson Plan

1

What is templates in HTML and why we need it

2

Twig template language

3

Templating features:

- variables
- cycles
- conditions
- include
- extends

Templating is a way to automate HTML

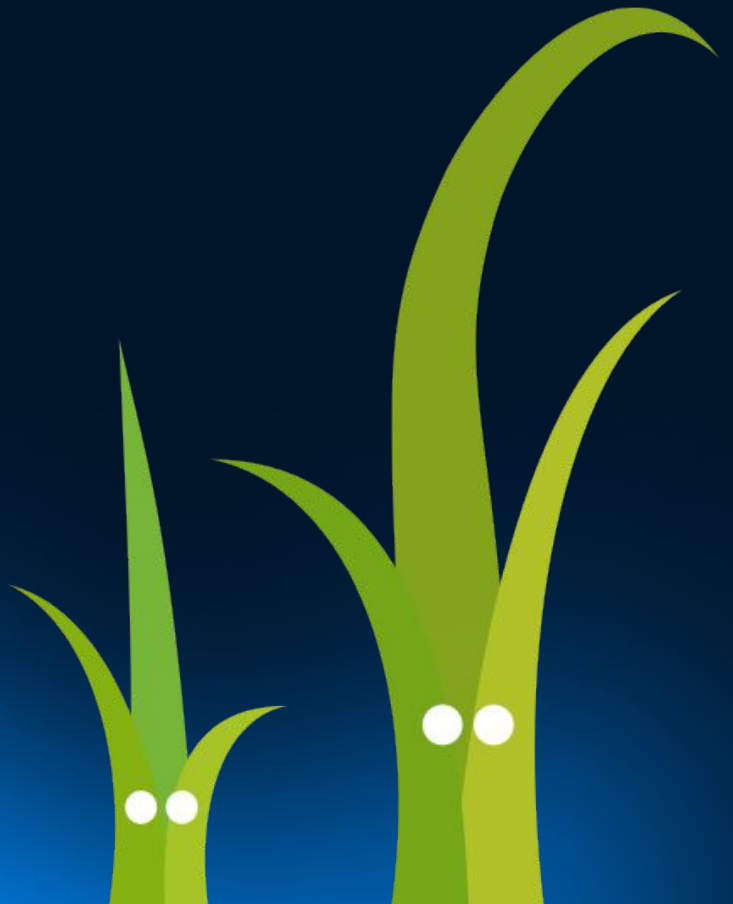
- helps to avoid code duplication
- simplifies code maintenance
- supports providing dynamic data

```
{% extends "base.html" %}

{% block header %}
<h1>{{ title }}</h1>
{% endblock %}

{% block content %}
<ul>
  {% for name, item in items %}
  <li>{{ name }}: {{ item }}</li>
  {% endfor %}
</ul>
{% endblock %}
```

Twig

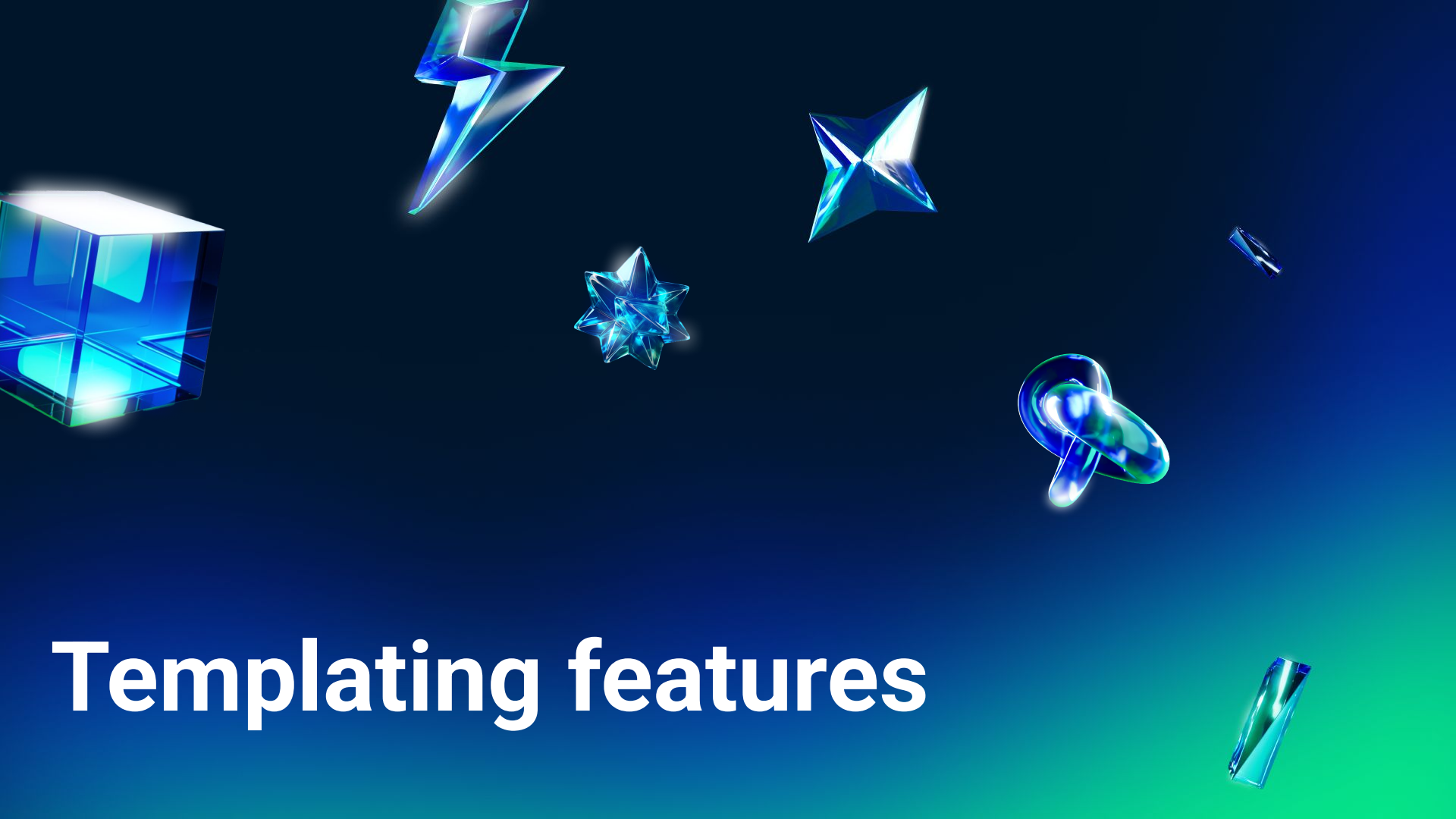


Twig is a templating language

- Reuse parts of UI as components
- Reuse and easily control a content with variables
- Repeat repetitive things with cycles
- Show different things depending on data

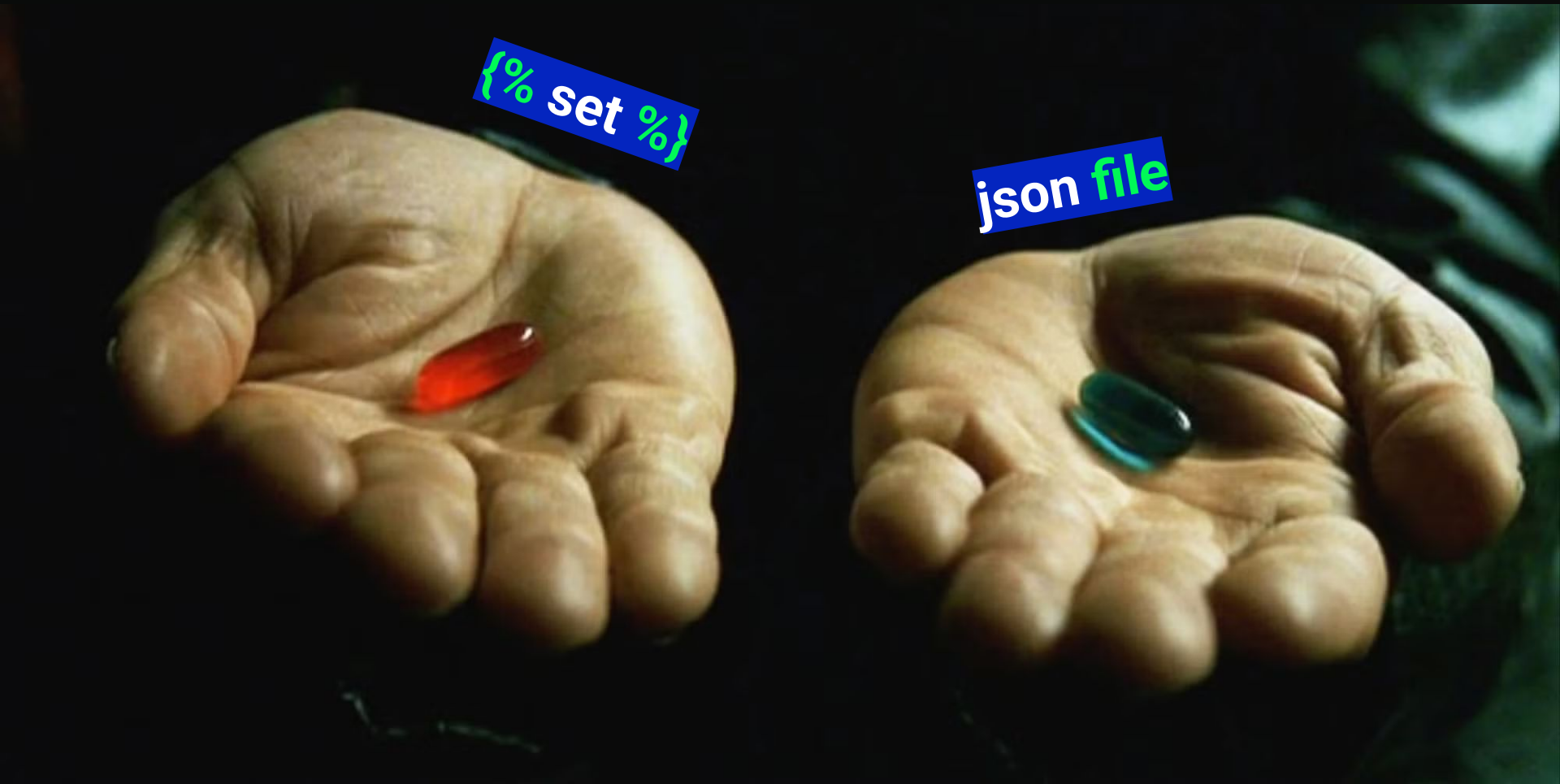
Twig in file system

```
├── src/
│   ├── pages/      # Pages (index.twig, blog.twig)
│   ├── templates/      # Folder with all templates
│   │   ├── layouts/    # Layouts (base.twig)
│   │   ├── partials/   # Repeating parts (header.twig, card.twig)
```



Templating features

Variables


{% set %}
json file

{% set %} a variable

and then you'll have an ability to use it somewhere else

in a twig file

```
{% set title = "Test page" %}
```

{{ variables }} are containers for data

You may use it to substitute a predefined data
on the place where you used a variable

in a twig file

```
{% set title = "Test page" %}
{{ title }}
```

2

Content from data.json

```
├── src/
│   ├── data/
│   │   │   ├── common.json
│   │   │   ├── index.json
```

1. File should be started with open curly bracket
2. and ended with closing curly bracket
3. Each object key and value should be in quotes*
4. Keys and values should be separated by colon

src/data/index.json

```
{
    "key": "value",
    "page": {
        "title": "Binabox",
        "catalog": {
            "title": "Catalog",
            "items": [
                {
                    "title": "$1990",
                    "text": "Echo of Bones",
                    "tag": {
                        "id": "new",
                        "title": "New",
                    }
                }
            ],
        },
    },
}
```

Types of values

String

```
{
    "key": "value"
}
```

String

```
{
    "pokemon_name": "Pikachu"
}
```

Object

```
{
    "key": {}
}
```

Object

```
{
    "pokemon": {
        "name": "Pikachu",
        "type": "Electric",
    }
}
```

Array

```
{
    "key": []
}
```

Types of values

String

```
{
    "key": "value"
}
```

Object

```
{
    "key": {}
}
```

Array

```
{
    "key": []
}
```

Array

```
{
    "pokemons": [
        {
            "name": "Pikachu",
            "type": "Electric",
        },
        {
            "name": "Bulbasaur",
            "type": "Grass",
        },
        {
            "name": "Charmander",
            "type": "Fire",
        }

}
```

Types of values

String

```
{
    "key": "value"
}
```

Object

```
{
    "key": {}
}
```

Array

```
{
    "key": []
}
```

src/data/index.json

```
{
    "page": {
        "title": "Binabox",
        "catalog": {
            "title": "Catalog",
            "items": [
                {
                    "title": "$1990",
                    "text": "Echo of Bones",
                    "tag": {
                        "id": "new",
                        "title": "New",
                    }
                }
            ],
        },
    },
}
```

Content from json file

src/data/index.json

```
{
     "index" : {
          "catalog_slider": {
               "title": "Catalog",
               "button": "See all"
          }
     }
}
```

Use _ (underscore)
Not - (dash)
to separate words

/src/templates/partials/catalog-slider.twig

```
<h2 class="catalog__title">
    {{ catalog_slider.title }}
</h2>
```

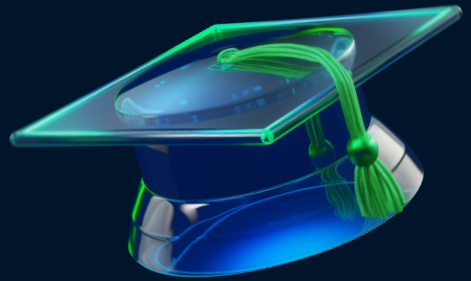
↓ npm run build

will be compiled into HTML as

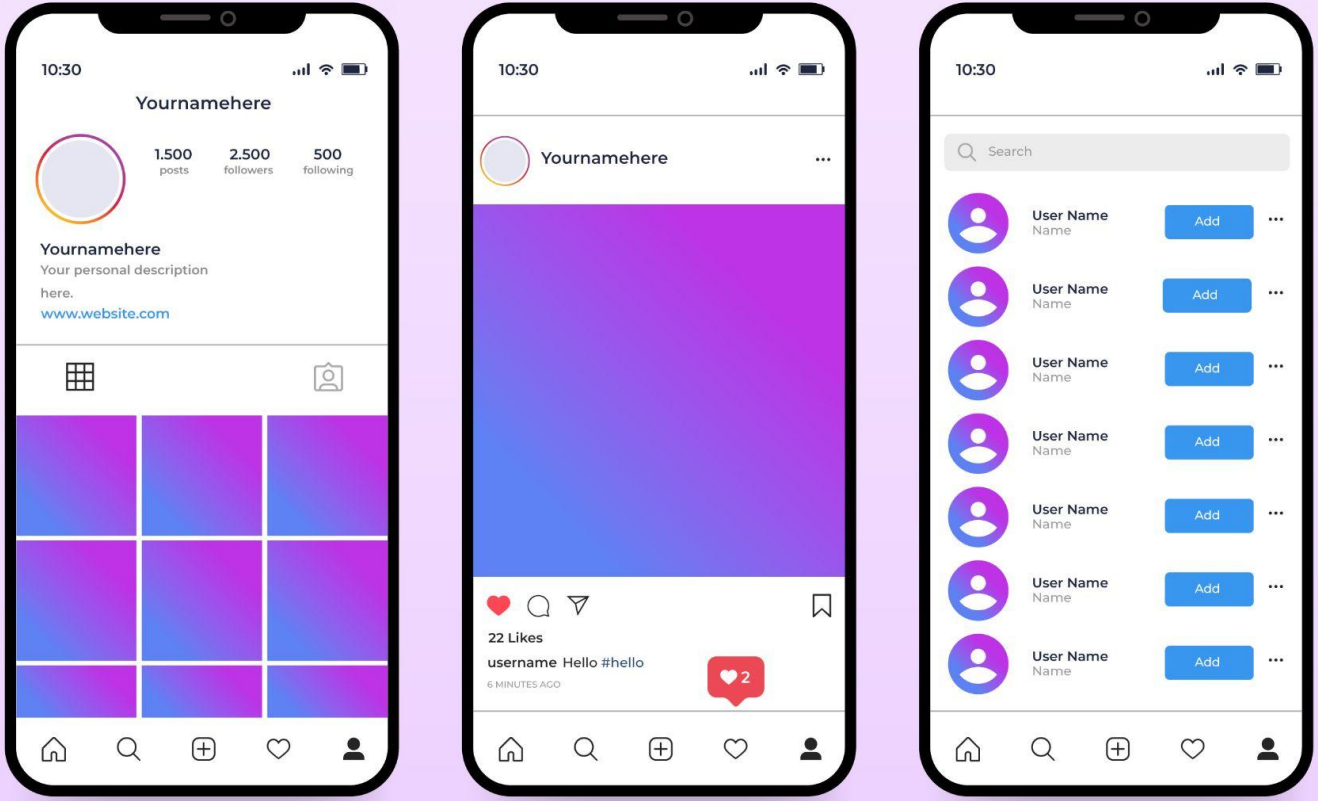
```
<h2 class="catalog__title">Catalog</h2>
```

When you need to use variables in real world

- when you’re getting data from different sources
- when data may be changed depending on any conditions and should be replaced automatically in different places
- or always to separate data from views and add make markup logic more transparent



Cycles


10:30
Yournamehere
1.500
posts
2.500
followers
500
following
Yournamehere
Your personal description
here.
www.website.com
10:30
Yournamehere
22 Likes
username Hello #hello
6 MINUTES AGO
2
10:30
Search
User Name
Name
Add
User Name
Name
Add
User Name
Name
Add
User Name
Name
Add
User Name
Name
Add
User Name
Name
Add
User Name
Name
Add

{% for %} cycle over arrays

will allow you to repeat HTML block
with different arrays of data

src/data/index.json

```
{
      "index" : {
            "networks": [
                  "facebook",
                  "telegram",
                  "instagram"
            ]
      }
}
```

/src/pages/index.twig

```
{% for icon in networks %}
    <img src="{{ icon }}-2x.webp">
{% endfor %}
```

/dist/index.html

```
<img src="facebook-2x.webp">
<img src="telegram-2x.webp">
<img src="instagram-2x.webp">
```

Our team

Jason Smith
Co - Founder

Jason Smith
Co - Founder

Jason Smith
Co - Founder

Jason Smith
Co - Founder

Jason Smith
Co - Founder

Jason Smith
Co - Founder

Jason Smith
Co - Founder

{% for %} with nested objects

/src/pages/index.twig

```
{% for person in index.team %}
- {{ person.name }}
- {{ person.role }}
- {{ person.avatar }}
{% endfor %}
```

/dist/index.html

```
- Jason Smith
- Co-Founder
- jason-smith.jpg
- Anna Klaus
- Engineer
- anna-klaus.jpg
```

src/data/index.json

```
{
    "index" : {
        "team" : [
            {
                "name": "Jason Smith",
                "role": "Co-Founder",
                "avatar": "jason-smith.jpg",
            },
            {
                "name": "Anna Klaus",
                "role": "Engineer",
                "avatar": "anna-klaus.jpg",
            },
        ]
    }
}
```

Home Blog Catalog

{% for %} with nested objects

in a twig file

```
<ul>
{% for item in nav %}
    <li>
        <a href="{{ item.url }}">
            {{ item.text }}
        </a>
    </li>
{% endfor %}
</ul>
```

will be compiled into HTML as

```
<ul>
<li><a href="/">Home</a></li>
<li><a href="/blog/">Blog</a></li>
<li><a href="/catalog/">Catalog</a></li>
</ul>
```

/src/data/common.json

```
{
    "common": {
        "nav": [
            {
                "url": "/",
                "text": "Home"
            },
            {
                "url": "/blog/",
                "text": "Blog"
            },
            {
                "url": "/catalog/",
                "text": "Catalog"
            }
        ]
    }
}
```

{% for %} with objects unpacking

in a twig file

```
<ul>
{% for url, text in nav %}
<li><a href="{{ url }}">{{ text }}</a></li>
{% endfor %}
</ul>
```

will be compiled into HTML as

```
<ul>
<li><a href="/">Home</a></li>
<li><a href="/blog/">Blog</a></li>
<li><a href="/catalog/">Catalog</a></li>
</ul>
```

/src/data/common.json

```
{
    "common": {
        "nav": [
            {
                "url": "/",
                "text": "Home"
            },
            {
                "url": "/blog/",
                "text": "Blog"
            },
            {
                "url": "/catalog/",
                "text": "Catalog"
            },
        ]
    }
}
```

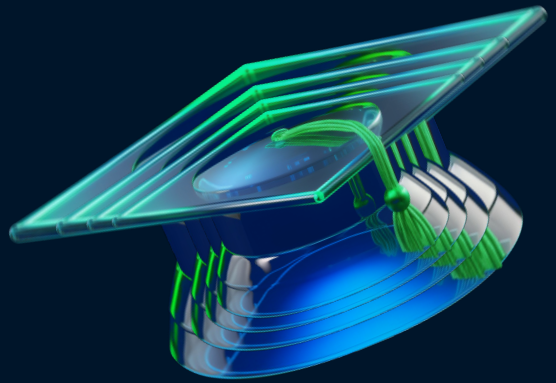
Home Blog Catalog

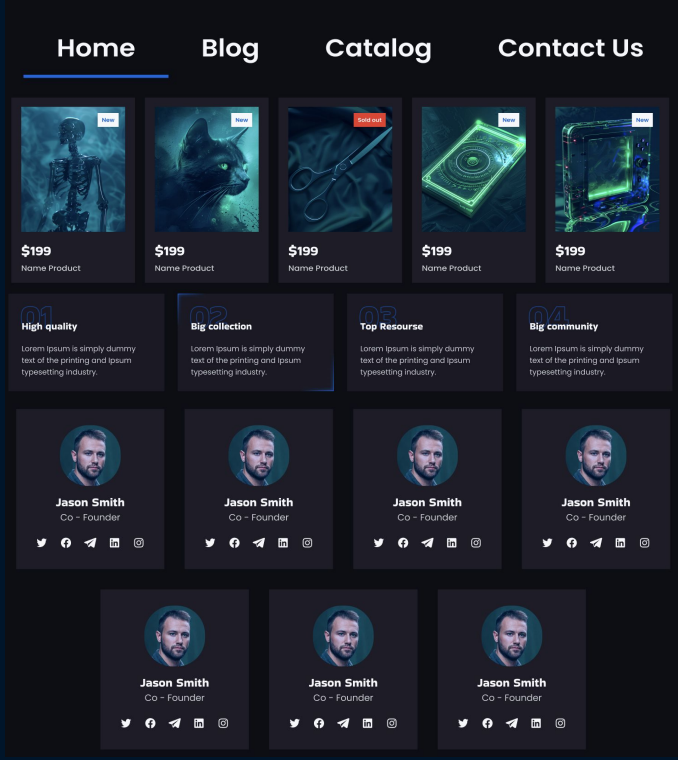
When you need to use cycles

- when you have repetitive content with typical design
- when items amount is known
- to not write annoying repetitive code

Examples:

- navigation items
- cards
- social icons
- accordion items
- categories list
- article tags
- catalog filter list
- etc





Conditions

{% if %} condition {% endif %}

check if variable is equal to something another

```
{% if something %}
    something
{% endif %}
```

```
{% if something == another %}
    something
{% endif %}
```

{% if %} condition {% endif %}

check if variable is equal to something

in a twig file

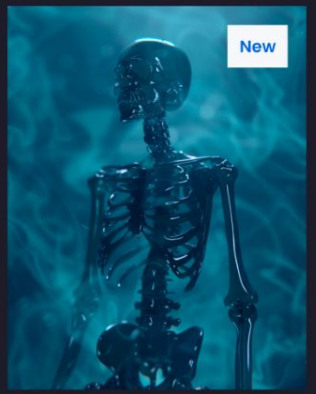
```
{% for item in nav %}
<a href="{{ item.url }}"
class="nav__link {% if nav.url == url
%}nav__link--active{% endif %}">
{{ item.text }}</a>
{% endfor %}
```

will be compiled into HTML as

```
<li><a class="nav__link"
href="/">Home</a></li>
<li><a href="/blog/" class="nav__link
nav__link--active">Blog</a></li>
<li><a class="nav__link"
href="/catalog/">Catalog</a></li>
```

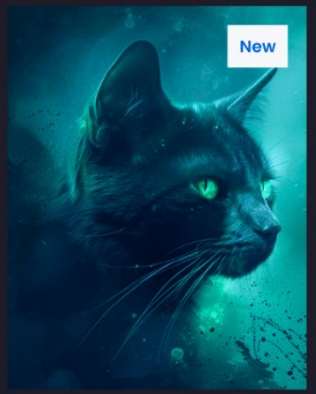
/src/data/common.json

```
{
    "common": {
        "nav": [
            {
                "url": "/",
                "text": "Home"
            },
            {
                "url": "/blog/",
                "text": "Blog"
            },
            {
                "url": "/catalog/",
                "text": "Catalog"
            }
        ]
    }
}
```



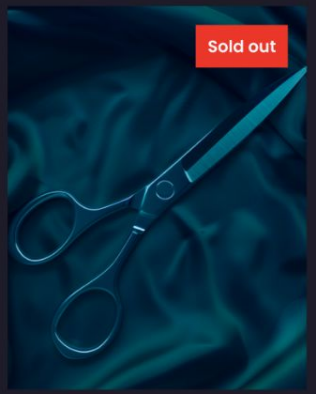
$199

Name Product



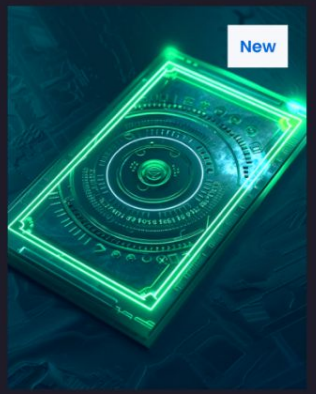
$199

Name Product



$199

Name Product

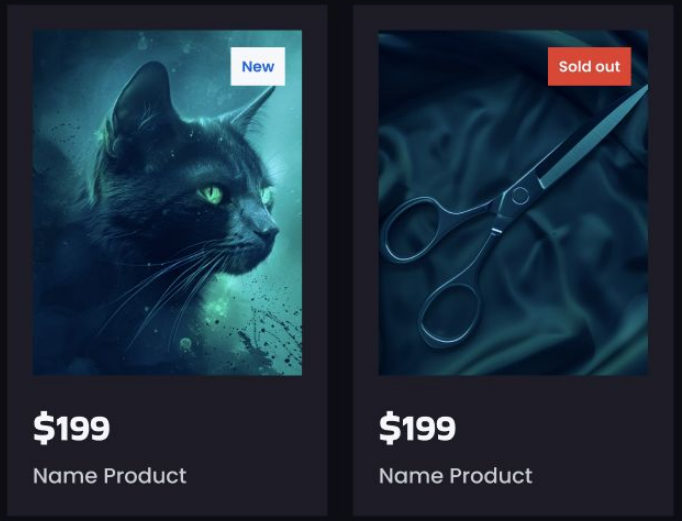


$199

Name Product

$199

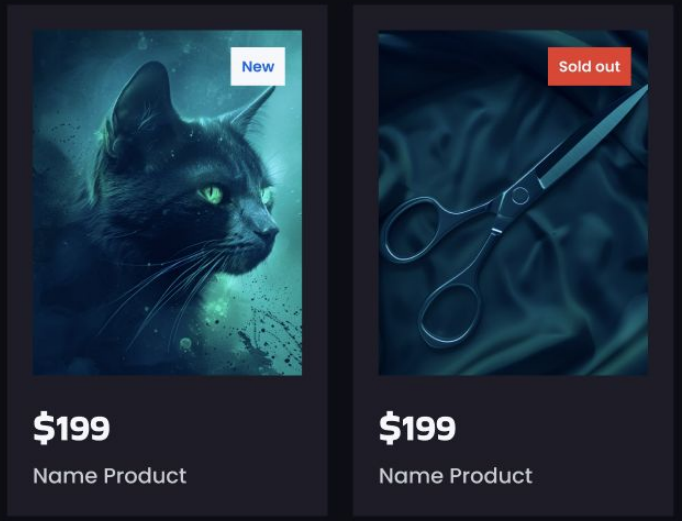
Name Product



/src/data/index.json

```
"catalog-items": [
    {
        "label": "New",
        "name": "Name Product",
        "price": "199",
        "image: "cat.jpg",
    },
    {
        "label": "Sold out",
        "name": "Name Product",
        "price": "199",
        "image: "cat.jpg",
    }
]
```

```
<p class="
    card__label
    {% if catalog-item.label == "Sold out" %}
        card__label--active
    {% endif %}"
>{{ catalog-item.label }}</p>
```



/src/data/index.json

```
"items": [
    {
        "label": "New",
        "state": "new",
        "price": "199",
        "name" : "Name Product",
        "image : "cat.jpg",
    },
    {
        "label": "Sold out",
        "state": "sold-out",
        "price": "199",
        "name" : "Name Product",
        "image : "cat.jpg",
    }
]
```

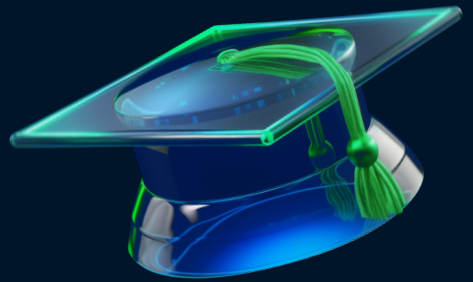
```
<p class="card__label card__label--{{ item.state }}">{{ item.label }}</p>
```

When you need to use condition

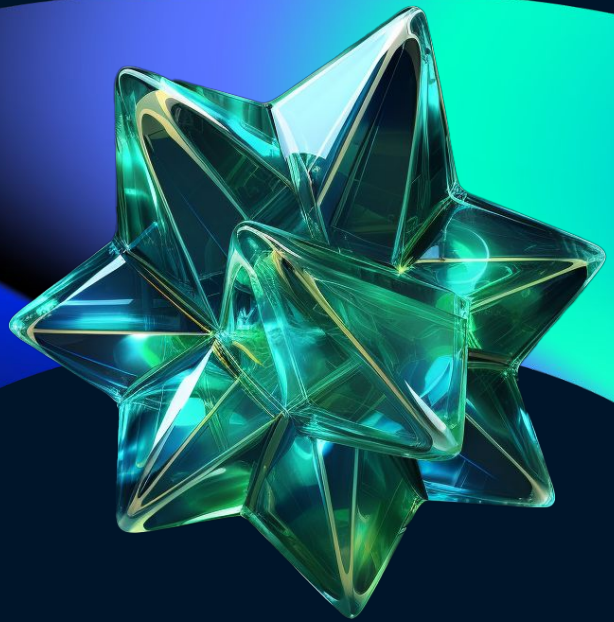
- when something should be changed in different cases

Examples:

- Highlight current page in the navigation menu
- Choose current radio button in filters list
- Show different tag on a card



Wednesday



Repeat

```
{% for item in nav %}
    <a href="{{ item.url }}"
        {% if nav.url == url %}
            class="active"
        {% endif %}
    >{{ item.text }}</a>
{% endfor %}
```

```
<li><a href="/">Home</a></li>
<li><a href="/blog/" class="active">Blog</a></li>
<li><a href="/catalog/">Catalog</a></li>
```

```
{
    "nav": [
        {
            "url": "/",
            "text": "Home"
        },
        {
            "url": "/blog/",
            "text": "Blog"
        },
        {
            "url":
        "/catalog/",
            "text": "Catalog"
        }
    ]
}
```

Includes

{% include %}

allows to put a template on desired place

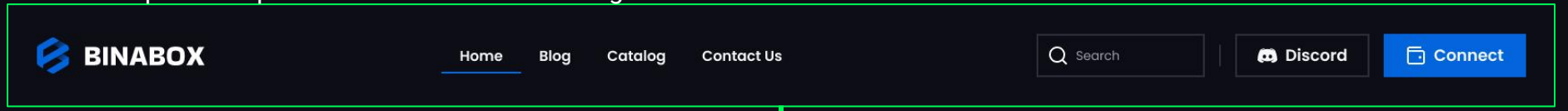
/src/templates/layout/base.njk

```
{% for item in nav %}
<a href="{{ item.url }}" class="nav__link {% if nav.url == url
%}nav__link--active{% endif %}">
{{ item.text }}</a>
{% endfor %}
```

/dist/blog.html

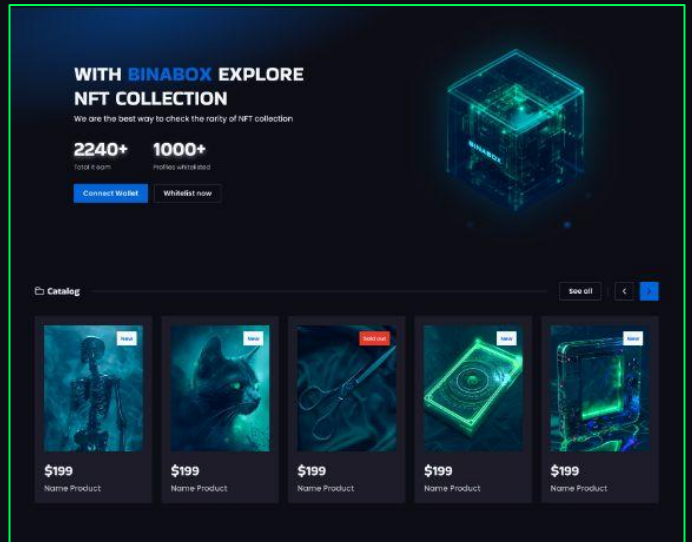
```
<li><a href="/">Home</a></li>
<li><a href="/blog/" class="active">Blog</a></li>
<li><a href="/catalog/">Catalog</a></li>
```

/src/templates/partials/ui/header.twig




```
{% include "/src/templates/partials/ui/header.twig" %}
```

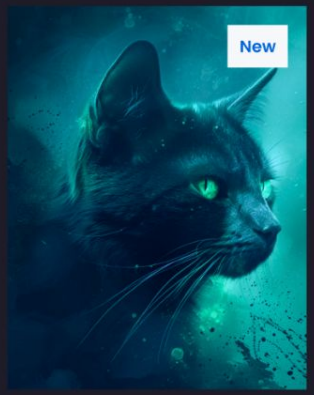
/src/templates/layouts/base.twig



```
{% for item in nav %}
    {% include "/src/templates/partials/card.twig" %}
{% endfor %}
```

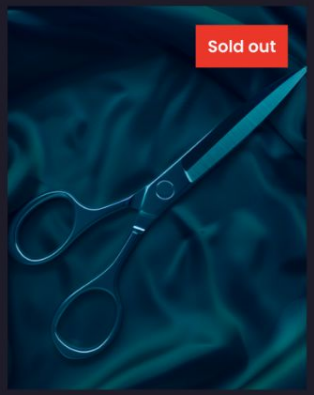
$199

Name Product



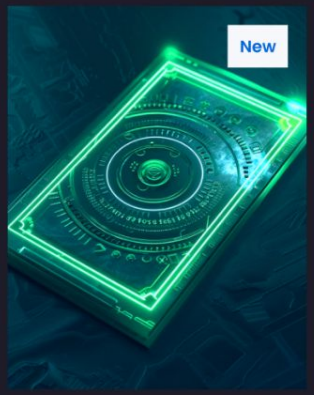
$199

Name Product



$199

Name Product



$199

Name Product

$199

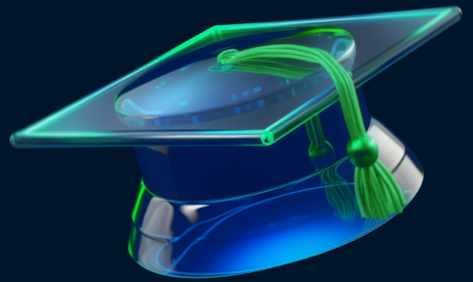
Name Product

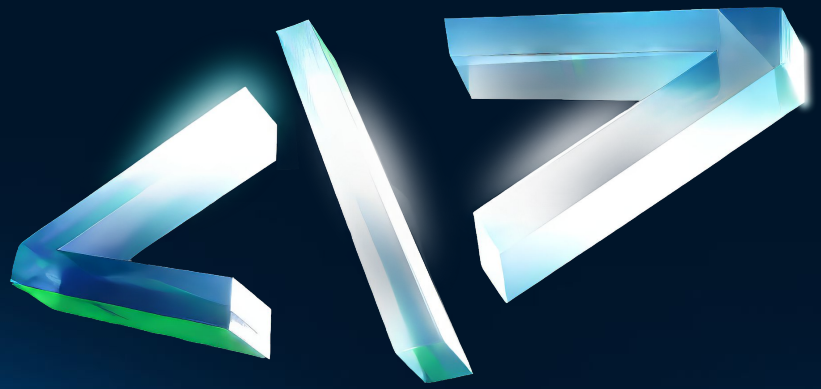
When you may want to use includes

- to reduce complexity of a component
- to name a component like a bem block

Examples:

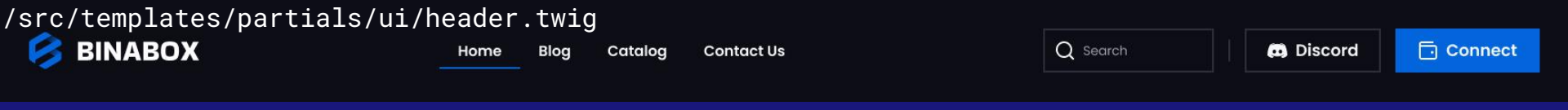
- header
- navigation
- card
- section





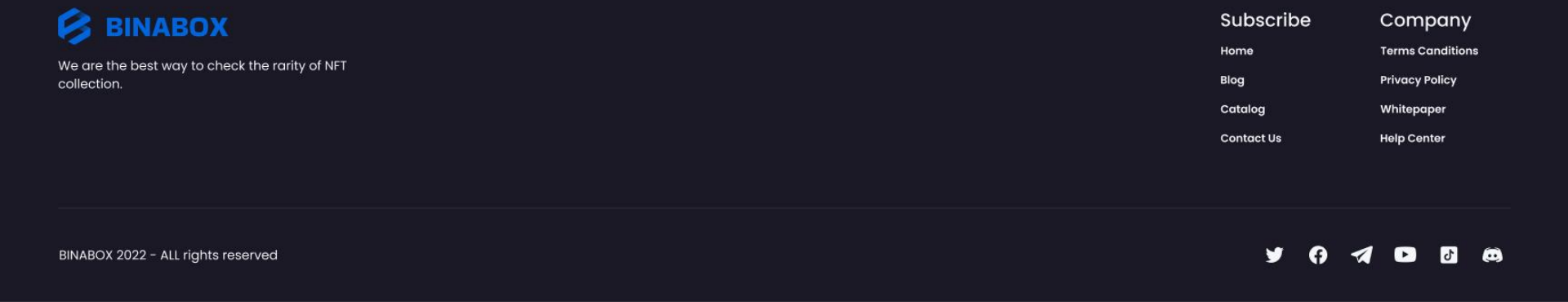
Extend layouts

/src/templates/partials/ui/header.twig




```
{% block "content" %}<h1>base</h1>{% endblock %}
```

/src/templates/partials/ui/footer.twig



{% extends %} and {% block %}

replacing blocks by names in a template

/src/templates/layouts/base.twig

```
<body>
{% block "content" %}<h1>base</h1>{% endblock %}
</body>
```

/src/pages/index.twig

```
{% extends "src/templates/layouts/base.twig" %}
{% block "content" %}<h1>index</h1>{% endblock %}
```

↓

/dist/index.html

```
<body>
<h1>index<h1>
</body>
```

/src/templates/pages/blog/blog.njk

```
{% extends "src/templates/layouts/base.njk" %}
any text
```

↓

/dist/blog.html

```
<body>
<h1>base</h1>
</body>
```

Pass page data throw layout into includes

/src/templates/layouts/base.njk

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{{ title or "Binabox" }}</title>
</head>
<body>
{% include "partials/header.twig" %}
{% block "content" %}<h1>base</h1>{% endblock %}
{% include "partials/footer.twig" %}
</body>
</html>
```

/src/templates/pages/blog/blog.twig

```
{% extends "layouts/base.twig" %}
{% block "content" %}<h1>blog</h1>{% endblock %}
```

/src/data/blog.json

```
{
    "title": "Blog"
}
```

/dist/blog.html

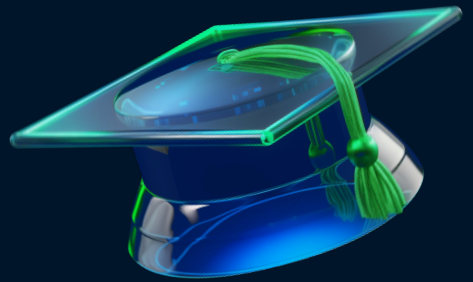
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Blog</title>
</head>
<body>
…header
<h1>blog</h1>
…footer
</body>
</html>
```

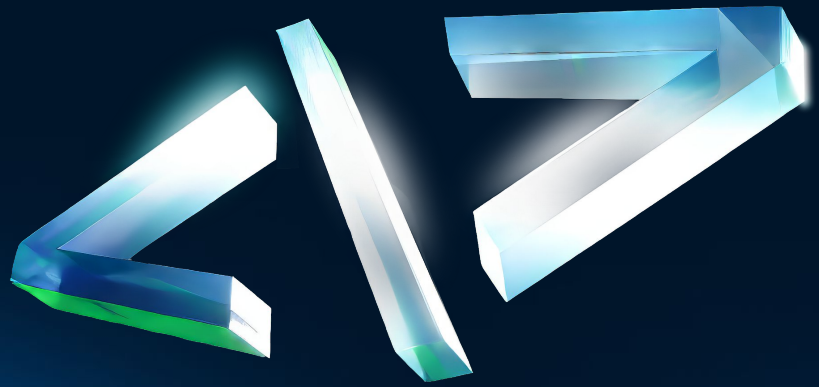
When you may want to use extends

when you need to have some places that should be used in other parts of your code

Examples:

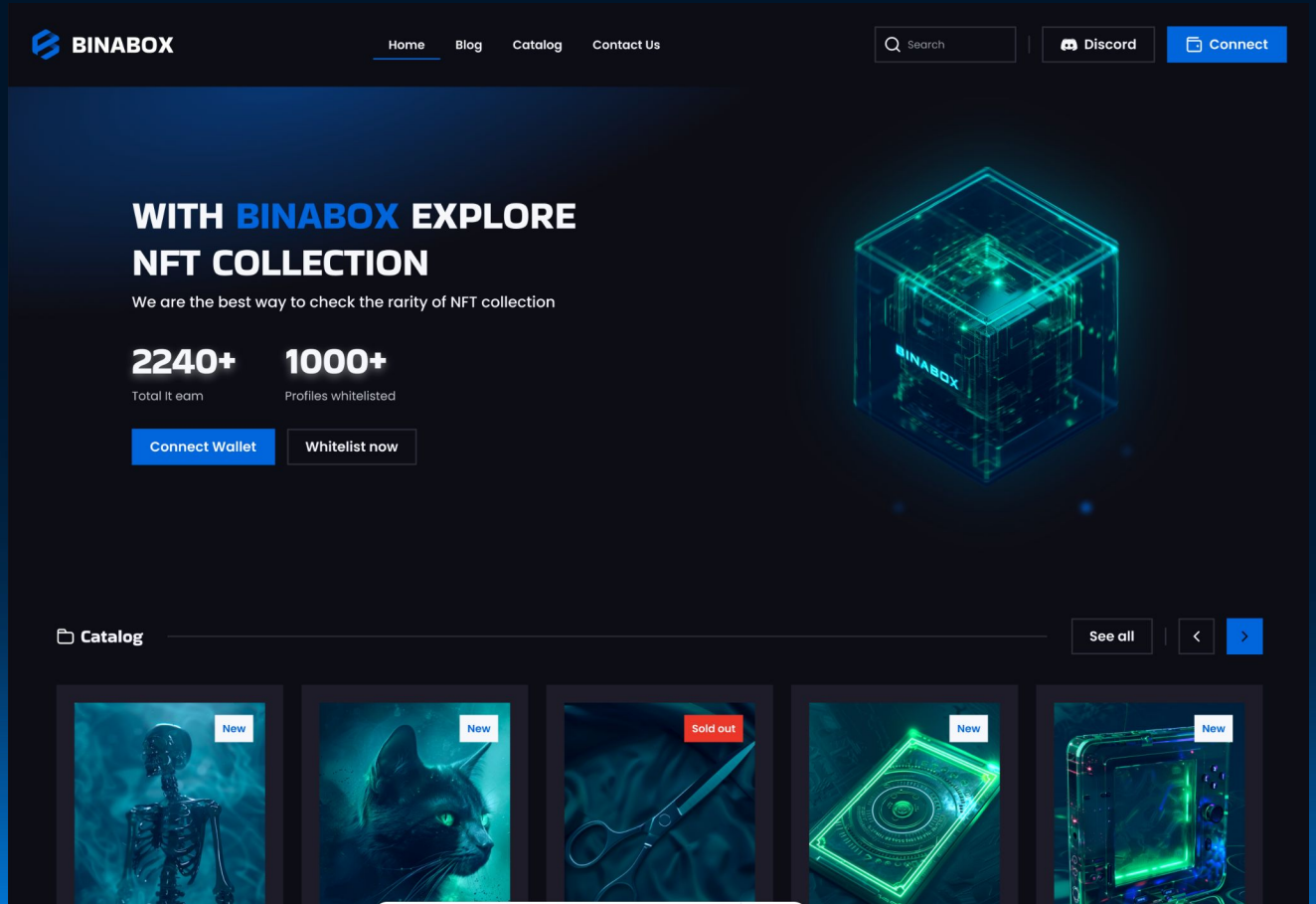
- layouts
- cards



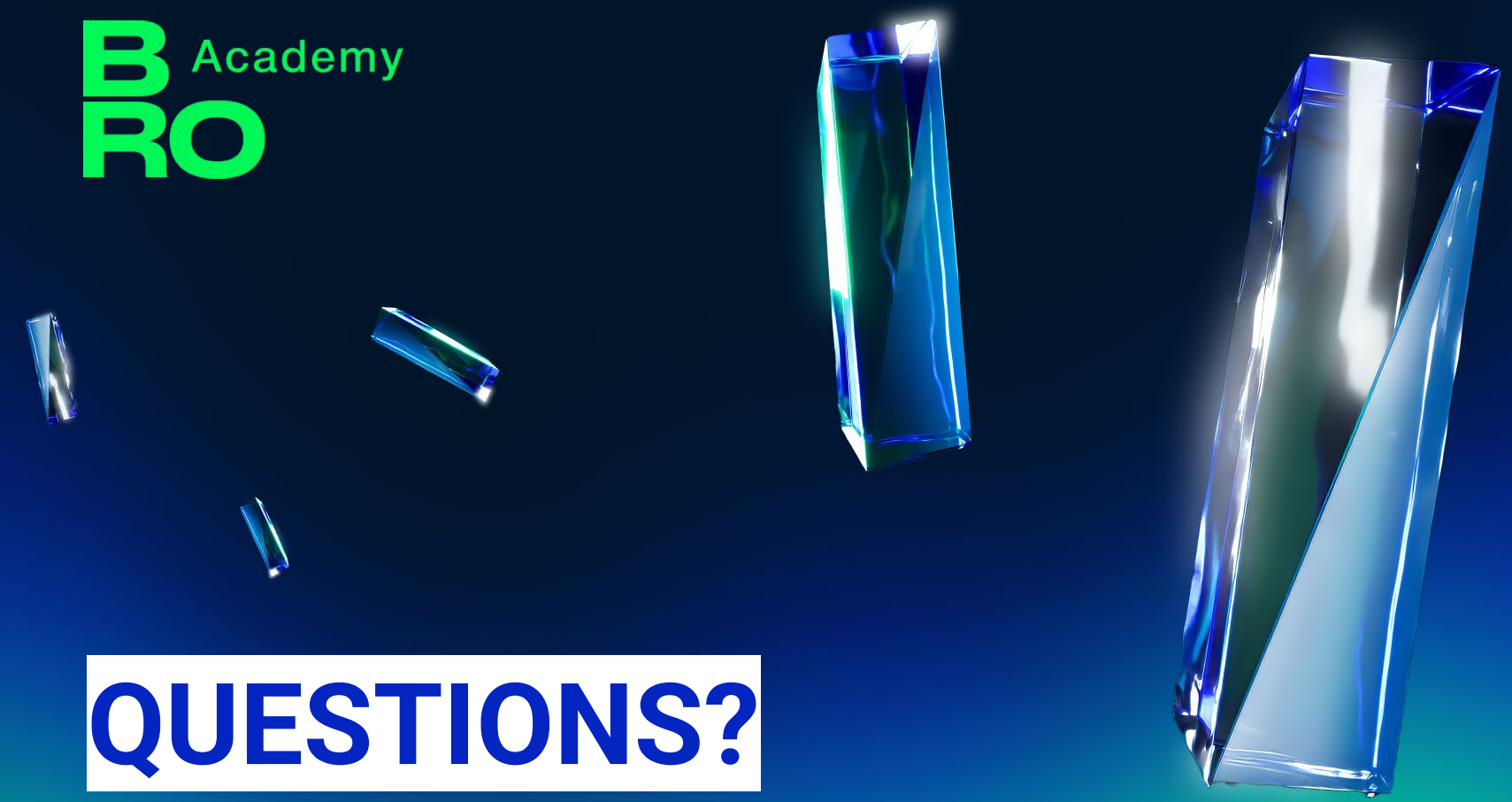


Summarize

Let’s check figma



BRO Academy



QUESTIONS?

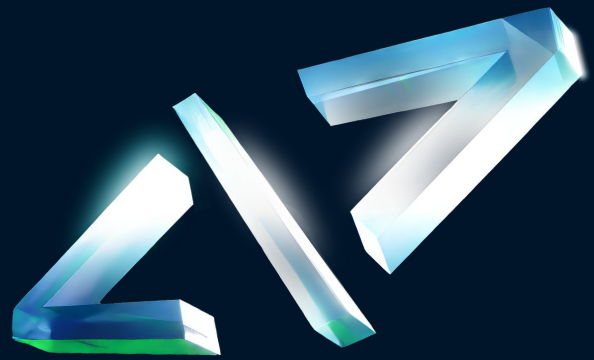
Homework

Part 1:Set Up Gulp Build

Copy the necessary Gulp configuration files (gulpfile.js, package.json, etc.) to your project.

Organize your project files:

- Move all images to the src/images/ folder.
- Copy the styles from your existing CSS file and place them in src/styles/style.scss.

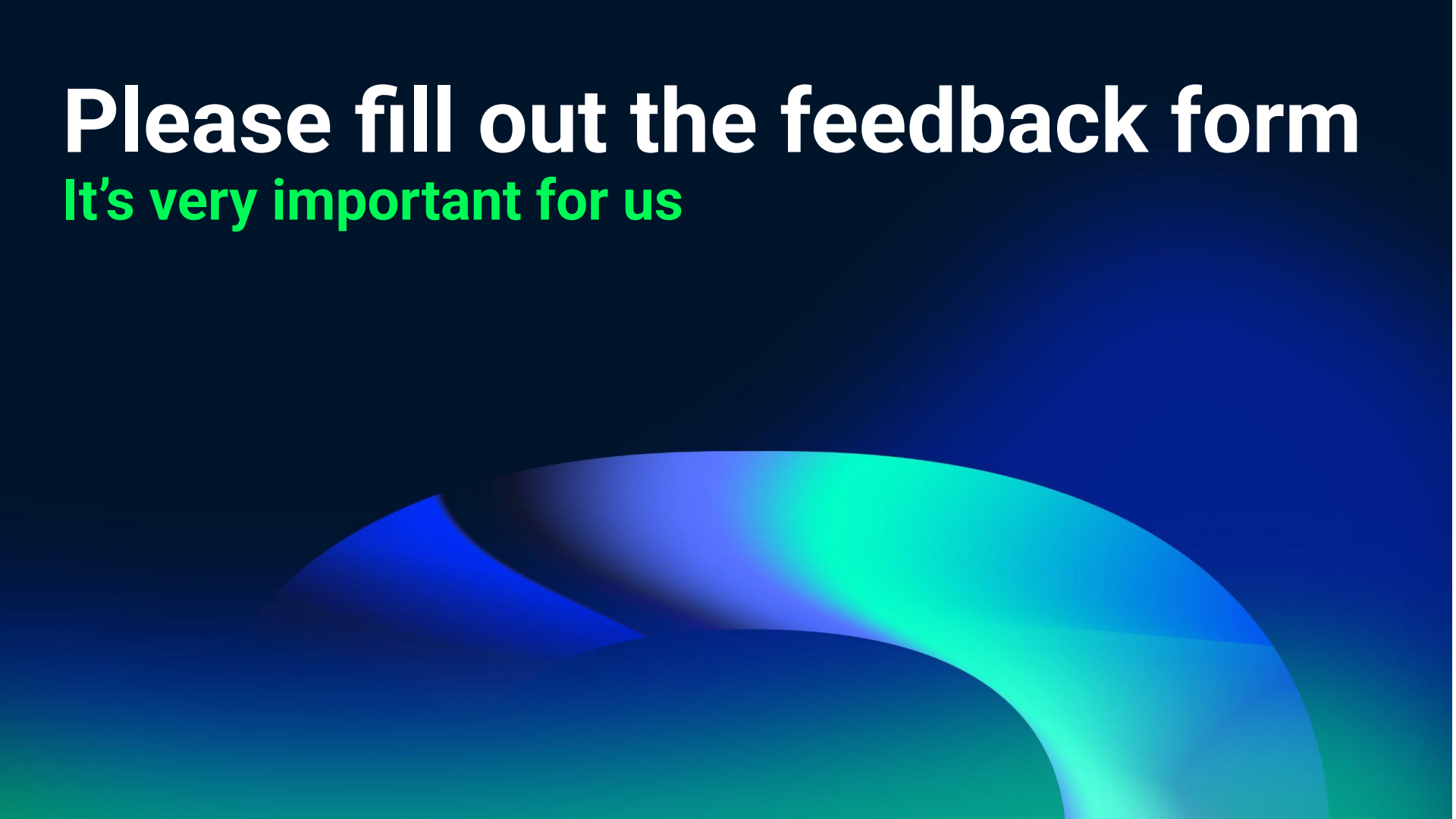


Create HTML Templates Using Twig

1. Set up the templates folder structure:
    - `src/templates/` → Main folder for all `.twig` files.
    - `src/templates/layouts/` → Base structure for pages (`base.twig`).
    - `src/templates/pages/` → Separate folder for each page (`index.twig`, `blog.twig`, etc.).
    - `src/templates/partials/` → Components (e.g., header, footer, product cards).
    - `src/data/data.json` → Store dynamic data (e.g., product lists, user profiles).
2. Implement the Nunjucks templating system:
    - Create a base template (`base.twig`) with `{% block %}` sections for content.
    - Use `{% extends %}` to inherit the layout structure.
    - Include reusable components (header, footer) with `{% include %}`.
    - Work with variables and loops (`{% for %}`, `{% if %}`) to generate dynamic content.

Please fill out the feedback form

It’s very important for us




THANK YOU!
Have a good evening!